Brickts SQL course — Week 12 — Read Open-FDD with confidence

sparql-playground / lessons/sql

Generated for offline / Kindle reading

Week 12 — Read Open-FDD with confidence

This pack concatenates the six daily lessons and the weekly challenge, then the Week primary-reading links from SOURCES.md. Work the SQL in `psql` against `brickts_course`; keep notes under ignored `student-work/sql/`.

Contents

- Day 78: Day 78 — Trace the production SQL input boundary
- Day 79: Day 79 — Inventory all 77 rules and the separate analytics pack

- Day 80: Day 80 — Audit and exercise the complete FC1 diagnostic
- Day 81: Day 81 — Separate a SQL metric from a runtime analytics query
- Day 82: Day 82 — Package a portable API and Plotly figure with provenance
- Day 83: Day 83 — Review a migration and an execution test plan
- Day 84 (weekly challenge): Day 84 — Independent capstone — specify model-driven SQL FDD and analytics

---

Day 78 — Trace the production SQL input boundary

Previous: Day 77 · **Week 12 · 45–75 minutes.** Prerequisites: Days 1–77, or equivalent SQL skills.

Goal

Read Open-FDD's schemas and execution adapter and distinguish source templates from the SQL actually executed.

Before you start

Open the pinned 3.6.15 references in Open-FDD bridge. Treat the production repository as read-only. The course's Brick profile and Open-FDD's Haystack model have separate identities and role vocabularies; use an explicit adapter.

Concept

The production long sample key is site, `openfdd_key` and timestamp. Its SQL input adapter builds a per-run temporary `history` relation with equipment ID, timestamp and role columns. The helper schema marker does not create that wide history itself. Trace `ensure_history_table`, `fill_history_for_equipment`, rendering and execution in `timescale_product.rs`. Compare with registry metadata and PostgreSQL helper functions. Numeric `{{...}}` rendering belongs to trusted reviewed templates with validated tuners; bound `$1` values belong to the execution interface. They serve different purposes and require separate validation.

Tiny example

```
SELECT site_id, equipment_id,
       ts AS timestamp_utc,
       fan_speed_pct / 100.0 AS fan_cmd,
       fan_status
FROM course.ahu_5m
WHERE site_id = 'SYN-NORTH'
  AND ts >= '2026-01-05
         08:00Z'::timestamptz
  AND ts < '2026-01-05
         08:15Z'::timestamptz
ORDER BY equipment_id, timestamp_utc;
```

The explicit percent-to-fraction conversion is safe because the course unit is declared. Renaming a pressure column does not convert Pa into a catalog tuner's in.w.c.

SQL challenge

Create a mapping document for all fourteen course roles: source unit, target role/unit, conversion, null/quality behavior and applicability. Build a scoped scratch view for FC1's required and optional inputs. Resolve UUIDs through metadata; define how duplicate same-role sensors would be rejected or selected. Identify the minimum helper dependencies from the pinned SQL rather than installing an entire production schema.

Experiment

Trace one catalog rule from its registry path through the actual PostgreSQL path selection, template rendering, temporary-history population and query wrapper. Record where site, equipment, window, equipment kind and missing roles are enforced. Mark a boundary as unverified when the source evidence is incomplete.

Acceptance checks

- The trace identifies SQL input grain and actual execution engine.
- Roles and units have explicit conversions; unknown mappings remain unsupported.
- The fixture adapter keeps bad/missing input null.
- No production SQL is executed against application data.

Stretch and reflection

Why can a permanent wide helper name in a comment mislead a reader about runtime behavior?

Save your SQL, actual output, and one explanation in `student-work/sql/day78/`. Use only `brickts_course`; see this week's primary

references.

Previous: Day 77 ·

Day 79 — Inventory all 77 rules and the separate analytics pack

Previous: Day 78 · **Week 12 · 45–75 minutes.** Prerequisites: Days 1–78, or equivalent SQL skills.

Goal

Make applicability and output type inspectable for the complete pinned FDD catalog.

Before you start

Use the pinned registry and the complete inventory in Open-FDD bridge. Inventory all 77 rule entries; a few familiar AHU rules are insufficient. Read the separate analytics registry without counting its query bodies as additional FDD rules.

Concept

A catalog carries rule IDs and aliases, equipment kinds, required/optional roles, output columns, tuners, units and defaults. These fields are an execution contract. Missing required roles differ from optional columns filled with null, and metadata existence differs from usable sample coverage. Some entries produce metrics rather than fault hours. Current runtime classification uses output columns; inspect it rather than assuming every catalog entry is a diagnostic. Catalog inclusion, dashboard wiring and successful qualification are separate claims.

Tiny example

```
WITH catalog(rule_id, output_columns) AS (
  VALUES ('FC1', ARRAY['equipment_id',
         'fault_hours']),
         ('FAN-RUNTIME-HOURS',
          ARRAY['equipment_id',
         'fan_runtime_hours',
         'total_hours'])
)
SELECT rule_id,
  output_columns && ARRAY['fault_hours',
         'fault_pct', 'fault_samples']
    AS diagnostic_shape
FROM catalog ORDER BY rule_id;
```

This reproduces a classification concept for two entries; it does not assert that those entries have passed execution tests.

SQL challenge

Import or transcribe the full pinned catalog into `student.rule_inventory`. Preserve aliases, SQL path, equipment kinds, required/optional roles, output columns, tuner names/types/ranges/units/defaults and the revision. Use a normalized tuner child table or structured JSON with validation. Produce counts by output shape and equipment kind, plus an applicability report for the course AHUs. Mark unsupported roles explicitly instead of manufacturing source data.

Experiment

Remove one required-role mapping for one scratch equipment item and contrast it with removal of an optional mapping. Record expected skip, null-column or ineligible behavior from the actual adapter. Inspect a non-AHU rule and explain why this course fixture cannot qualify it.

Acceptance checks

- There are exactly 77 distinct canonical rule IDs and no duplicate alias ownership.
- Inventory includes metric entries and every tuner's declared metadata.
- Applicability retains site/equipment scope and explicit missing roles.
- The 18 analytics query bodies have their own inventory and statuses.

Stretch and reflection

Which checks would detect a new registry entry whose executable SQL file was omitted from packaging?

Save your SQL, actual output, and one explanation in `student-work/sql/day79/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 78 ·

Day 80 — Audit and exercise the complete FC1 diagnostic

Previous: Day 79 · **Week 12 · 45–75 minutes.** Prerequisites: Days 1–79, or equivalent SQL skills.

Goal

Explain FC1's full pinned algorithm and verify its confirmation behavior using adapted synthetic input.

Before you start

Read FC1's PostgreSQL template, registry parameters and parameter derivation through Open-FDD bridge. Use a session-local temporary `history` populated only from scoped fixture data. Record conversions so pressure inputs and `EPS_DSP` share units.

Concept

FC1 includes operational gating, fan-status precedence, a full-fan gate, startup/mode delay, low-pressure comparison, raw-fault streaks, confirmation and a minimum-active-coverage rule. Its implementation counts confirmed rows times declared polling seconds. A row streak is not automatically elapsed-time continuity. Raising confirmation excludes early streak rows; the template does not retroactively credit the whole streak. Review null/fallback branches as authored. These are production-study semantics, not an endorsement that row counts suit irregular data.

Tiny example

```
WITH streak(raw_fault, streak_len) AS (
  VALUES (1, 1), (1, 2), (1, 3), (0, 1)
)
SELECT raw_fault, streak_len,
       CASE WHEN raw_fault = 1 AND
         streak_len >= 3
            THEN 1 ELSE 0 END AS confirmed
FROM streak ORDER BY raw_fault DESC,
         streak_len;
```

Only the third fault row is confirmed. This example isolates one mechanism; it is not FC1's complete predicate.

SQL challenge

Render and run the full pinned PostgreSQL FC1 template against north Supply A on January 5. Freeze all effective tuners and adapt role names/units explicitly. Keep the regular 288-row equipment timeline, including null pressure at the two missing instants. With default 300-second confirmation, expect ten confirmed five-minute rows: 50 observed fault minutes. With 900-second confirmation, two five-row raw streaks yield six confirmed rows: 30 minutes. Verify timestamp masks as well as totals. For mask evidence, use an audit copy that carries `timestamp_utc` through `final` and selects it with `confirmed`; retain the unchanged template for aggregate parity.

Experiment

In scratch input, set status off while command stays on, then make status null. Explain the changed precedence. Separately delete intervening timeline rows and demonstrate why adjacent-row confirmation needs a cadence/gap contract. Document differences from the course's bounded-duration predicate.

Acceptance checks

- Every FC1 gate and effective tuner is accounted for.
- Full-template outputs match independently hand-counted masks.
- Pressure units and percent/fraction handling are explicit.
- Missing roles, null input and insufficient coverage have declared statuses.

Stretch and reflection

How would you distinguish authored row-count parity from a proposed elapsed-time correction?

Save your SQL, actual output, and one explanation in `student-work/sql/day80/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 79 ·

Day 81 — Separate a SQL metric from a runtime analytics query

Previous: Day 80 · **Week 12 · 45–75 minutes.** Prerequisites: Days 1–80, or equivalent SQL skills.

Goal

Implement one metric with a known answer and review a similarly named production query for different semantics.

Before you start

Read pinned `fan_runtime_hours.sql`, the analytics `motor_weekly_runtime.sql`, and their actual callers through Open-FDD bridge. Use only course or handcrafted scratch input.

Concept

The catalog metric `FAN-RUNTIME-HOURS` uses non-null normalized `fan_cmd > 0.05`, then multiplies row counts by polling seconds. Its `total_hours` is eligible command-row

duration, not a guaranteed complete window. The separate runtime analytics query counts matching status and command sample rows, uses a fixed 300 seconds and binds site only. Both channels can count the same instant twice; `LIMIT 500` bounds returned groups without bounding historical scan work. Similar names do not establish equivalent equations. Review the source behavior before selecting a contract.

Tiny example

```
WITH input(channel, val) AS (
  VALUES ('fan_status', 1.0), ('fan_cmd',
         0.7)
)
SELECT count(*) FILTER (WHERE val > 0.05)
         * 300.0 / 3600.0
       AS row_count_hours
FROM input;
```

Two active channels yield one sixth of an hour from a single five-minute instant. A one-signal runtime definition would yield one twelfth. This is a counterexample to using multi-channel row counts as the oracle.

SQL challenge

Execute the full catalog metric in a scoped scratch environment after converting course `fan_speed_pct` to a fraction. For north Supply A on January 5, expect 12 command-on hours and 24 eligible timeline hours. Build a bounded analytics query over the same equipment/window with one explicitly selected authoritative signal. Include coverage and compare its result with the catalog metric. Any deliberate equation correction must have its own version and tests.

Experiment

Add status and command for the same instants to a handcrafted long table, then remove one channel. Contrast the production analytics equation with your chosen single-signal contract. Introduce a gap and explain the polling assumption separately from duplicate-channel effects.

Acceptance checks

- Both source query paths and their bind parameters are identified.
- A hand-derived counterexample detects double counting.

- Your query has site, identity and finite time bounds.
- Results declare units, denominator, authoritative signal and equation version.

Stretch and reflection

Which metric requires interval integration rather than multiplying sample counts by a nominal cadence?

Save your SQL, actual output, and one explanation in `student-work/sql/day81/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 80 ·

Day 82 — Package a portable API and Plotly figure with provenance

Previous: Day 81 · **Week 12 · 45–75 minutes.** Prerequisites: Days 1–81, or equivalent SQL skills.

Goal

Carry a consistent result cohort from SQL through a language-neutral API response into a small plot artifact.

Before you start

Reuse day59's contract and one day80 diagnostic or day81 metric. A JSON response and Plotly figure specification are sufficient; no running app, CDN or new UI is required. Keep all data synthetic.

Concept

A result cohort fixes site, equipment set, model revision, rule/query version, effective tuners, window and data snapshot. Summary

cards and plots must refer to that same cohort. A plot of raw samples may support diagnosis but cannot silently stand in for a confirmed-fault mask. Missing intervals need visible gaps. Plotly figure JSON uses `data` traces and `layout`; the API can supply this or a documented typed series contract without depending on one server language.

Tiny example

```
{
  "data": [{"type": "scatter", "mode":
        "lines+markers",
    "x": ["2026-01-05T08:20:00Z", "2026-
        01-05T08:25:00Z"],
    "y": [100, null], "connectgaps":
        false,
    "name": "Duct static (Pa)"}],
  "layout": {"yaxis": {"title": {"text":
        "Pa"}}}
}
```

This small figure preserves one explicit gap. Keep point identity and revision provenance in the surrounding response rather than relying on a human-readable trace name.

SQL challenge

Produce a bounded response for north Supply A's low-pressure hour with raw pressure/setpoint, confirmation mask,

coverage and summary. Include run/cohort ID, UTC half-open range, units, source point UUIDs, model/data revisions, SQL/template hash, effective tuners and result status. Define point/trace/payload caps. Prepare the same contract for a second server language without rewriting the SQL equation.

Experiment

Attempt to pair a day80 summary with a plot generated using different confirmation or window parameters. Show that cohort validation rejects the mismatch. Compare full-resolution mask evidence with a downsampled visual; explain which facts the latter cannot establish.

Acceptance checks

- Summary and figure share the same declared cohort.
- Aligned arrays, null gaps, units and UTC timestamps are valid.
- SQL values are bound and role identifiers come from a reviewed allowlist.
- Any downsampling method and loss of detail are disclosed.

Stretch and reflection

Which provenance fields should survive an exported figure so another reviewer can reproduce it?

Save your SQL, actual output, and one explanation in `student-work/sql/day82/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 81 ·

Day 83 — Review a migration and an execution test plan

Previous: Day 82 · **Week 12 · 45–75 minutes.** Prerequisites: Days 1–82, or equivalent SQL skills.

Goal

Prepare a concrete review packet for moving one model-driven diagnostic and one analytics query toward deployment.

Before you start

Use the preceding lessons' SQL, tests, manifests and contracts. Keep production read-only. This is a review and proposal exercise; deployment approval is outside the lab.

Concept

A migration changes installed behavior and data assumptions. A review packet should make identity, grain, units, scope, parameter

validation, applicability, cancellation, result typing and provenance inspectable. Source parity does not prove installed parity; equal totals do not prove equal timestamp masks. A controlled rollout needs success criteria, observability and a recovery plan compatible with any data changes. Record unanswered questions as specific tests, not generic assurance.

Tiny example

```
SELECT table_schema, table_name,
         column_name, data_type
FROM information_schema.columns
WHERE table_schema = 'student'
  AND table_name IN ('rule_inventory',
         'run_demo', 'result_demo')
ORDER BY table_name, ordinal_position;
```

Compare these actual objects with the manifest your proposal expects. An absent object is evidence of an incomplete local installation, even when its migration file exists.

SQL challenge

Write a migration proposal for your scoped SQL input adapter, versioned rule/analytics contracts and run/result storage. Include explicit columns, logical/composite keys,

indexing, installed-schema checks and least-privilege assumptions. Supply a test plan covering fresh install, applicable upgrade, wrong-site queries, duplicate same-role sensors, unit conversions, null/gap input, cancellation cleanup, full FC1 masks and the day81 channel counterexample. Identify required checks before any rollout.

Experiment

Review a deliberately flawed scratch change: remove a site component from a join, broaden a missing-equipment query, or copy only a run header. Use existing tests to expose the problem. Produce a concise code-review comment with the concrete trigger, wrong result and required behavior.

Acceptance checks

- The packet contains reviewable SQL/DDL and a test matrix with expected outcomes.
- Source, rendered SQL, installed schema and result provenance have distinct checks.
- Each unrun deployment requirement has an owner or explicit environment need.
- Performance evidence includes bounded scope and failure behavior.

- Recovery handles partial failures without presenting incomplete runs as complete.

Stretch and reflection

What evidence would justify promotion from a synthetic lab to a monitored pilot? Which requirements still need domain review?

Save your SQL, actual output, and one explanation in `student-work/sql/day83/`. Use only `brickts_course`; see this week's primary references.

Previous: Day 82 ·

Day 84 — Independent capstone — specify model-driven SQL FDD and analytics

Previous: Day 83 · **Week 12 review · 2–4 hours, split across sessions as needed.** Prerequisites: Days 78–83, or equivalent skills.

Project brief

Deliver a coherent specification and executable SQL evidence for model-driven FDD and analytics. The read path must resolve RDF metadata to opaque historian identity, adapt roles/units explicitly, execute a full diagnostic and a separate metric/analytics query, and package bounded results with reproducible provenance. Choose your design independently; any API language may implement the contract.

Use the synthetic course database and pinned references in Open-FDD bridge. This capstone demonstrates the proficiency

evidenced by your artifacts; it does not confer a degree or guarantee production readiness.

Acceptance criteria

- All 77 FDD entries and the separate analytics inventory have applicability and tuner/type metadata.
- One complete FC1 implementation and one metric/analytics query have independent known answers, timestamp/coverage evidence and negative tests.
- RDF resolution, site isolation, duplicate-role policy, unit conversion and immutable revisions are explicit.
- Parameterized SQL, bounded execution, pagination/aggregation, cancellation and resource budgets are reviewable.
- API/Plotly artifacts share a result cohort; schema/migration and qualification plans record remaining gaps honestly.

Deliverables

Submit architecture and contract documents, SQL/DDL, inventory, assertions, compact plans/performance evidence, sample responses/figure JSON and a qualification

ledger. Label observed, skipped, blocked and proposed work distinctly. Keep experiments within earlier limits.

Self-review

Trace one result from model to plot. Defend its units, duration and authorization. Explain one production-source defect you caught and one unresolved requirement that prevents a stronger claim.

Previous: Day 83 ·

Primary reading — Week 12

Use the lesson first, then these references for precise behavior. Full list: SOURCES.md.

Week 12

- Pinned Open-FDD production source map
- Generated 77 rule/18 analytics inventory
- PostgreSQL CREATE FUNCTION
- PostgreSQL testing concepts

References explain mechanisms, not engineering acceptance of an actual building. Do not copy copyrighted standards text or

claim a synthetic pass proves calibrated savings, real device compatibility or production security.